

Sprint 1 — Mobile Application Foundation

Boardroom Booking System — React Native Expo

Table of Contents

- [1. Overview](#)
- [2. Project Foundation](#)
- [3. Folder Structure](#)
- [4. Environment Configuration](#)
- [5. EAS Build Configuration](#)
- [6. Navigation Foundation](#)
- [7. Authentication Foundation](#)
- [8. API & Data Layer](#)
- [9. State Management Foundation](#)
- [10. Design System Foundation](#)
- [11. Reusable Component Library](#)
- [12. Form Framework](#)
- [13. Notification Framework](#)
- [14. Application Entry Point](#)
- [15. Authentication Flow — End to End](#)
- [16. Acceptance Criteria Checklist](#)

1. Overview

This document describes the implementation of Sprint 1 for the Internal Boardroom Booking Mobile Application. The sprint delivered a complete, production-ready React Native Expo foundation covering project structure, navigation, authentication, API integration, state management, design system, reusable components, form validation, and push notification infrastructure.

Technology Stack:

Layer	Technology
Framework	React Native (Expo SDK 56, managed workflow)
Language	TypeScript (strict mode)
Navigation	React Navigation v7
State Management	Redux Toolkit + RTK Query
Form Validation	react-hook-form + Zod v4
Persistence	AsyncStorage
Notifications	expo-notifications
Build	Expo EAS

2. Project Foundation

Application Configuration — `mobile/app.json`

The Expo application is configured with the following settings:

```
Name: Boardroom Booking
Slug: boardroom-booking
Version: 1.0.0
Orientation: portrait
Bundle ID: com.equisoft.boardroombooking (iOS & Android)
New Arch: enabled (React Native new architecture)
```

Platform-specific configuration:

- iOS:** Tablet support enabled
- Android:** Adaptive icon with separate foreground, background, and monochrome images. Permissions: `RECEIVE_BOOT_COMPLETED`, `VIBRATE`
- Web:** Favicon configured

Splash screen: Uses brand colour `#1E3A5F` as background with a contained splash image.

Expo Notifications plugin is registered at the app level with the brand icon and colour applied.

3. Folder Structure

The application follows a strict **feature-based architecture**. Each feature owns its screens, services, and models. Shared code lives in `shared/`. No business logic exists in presentation components.

```
mobile/
├── App.tsx                # Application entry point
├── app.json               # Expo app configuration
├── eas.json               # EAS build profiles
├── babel.config.js        # Babel configuration (path aliases)
├── tsconfig.json          # TypeScript configuration
├── index.ts               # Expo root component registration
├── .env                   # Local environment variables (gitignored)
├── .env.example           # Environment variable template
├── assets/                # App icons and splash images
├── src/
│   ├── api/              # API layer
│   │   ├── api.slice.ts  # RTK Query base API slice
│   │   ├── auth.api.ts   # Auth endpoints
│   │   └── error.utils.ts # Standardised API error extraction
│   ├── config/
│   │   └── env.ts        # Environment variable access
│   ├── design-system/    # Design tokens
│   │   ├── colors.ts
│   │   ├── typography.ts
│   │   ├── spacing.ts
│   │   ├── borders.ts
│   │   ├── shadows.ts
│   │   └── index.ts      # Barrel export
│   ├── features/
│   │   ├── auth/
│   │   │   ├── screens/
│   │   │   │   ├── LoginScreen.tsx
│   │   │   │   ├── ForgotPasswordScreen.tsx
│   │   │   │   └── ResetPasswordScreen.tsx
│   │   │   └── services/
│   │   │       ├── auth.service.ts      # Login, logout, session restore
│   │   │       └── auth-storage.service.ts # AsyncStorage token/profile ops
│   │   ├── dashboard/screens/DashboardScreen.tsx
│   │   ├── boardrooms/screens/BoardroomsScreen.tsx
│   │   ├── bookings/screens/BookingsScreen.tsx
│   │   ├── notifications/screens/NotificationsScreen.tsx
│   │   └── profile/screens/ProfileScreen.tsx
│   ├── navigation/
│   │   ├── RootNavigator.tsx # Auth-aware root navigator
│   │   ├── AuthStack.tsx     # Unauthenticated navigation stack
│   │   ├── MainTabs.tsx      # Authenticated bottom tab navigator
│   │   └── types.ts          # Navigation type definitions
│   ├── shared/
│   │   ├── components/
│   │   │   ├── buttons/      # PrimaryButton, SecondaryButton, DangerButton, IconButton
│   │   │   ├── inputs/       # TextInput, PasswordInput, SearchInput, SelectInput
│   │   │   ├── layout/       # ScreenContainer, PageHeader, SectionCard
│   │   │   ├── feedback/     # LoadingState, EmptyState, ErrorState
│   │   │   ├── modals/       # ConfirmationModal, SuccessModal, ErrorModal
│   │   │   └── index.ts      # Barrel export
│   │   ├── services/
│   │   │   └── notification.service.ts
│   │   ├── utils/
│   │   │   ├── validation.utils.ts
│   │   │   └── format.utils.ts
│   ├── store/
│   │   ├── index.ts          # Redux store configuration
│   │   ├── hooks.ts          # Typed useAppDispatch and useAppSelector
│   │   └── slices/
│   │       ├── auth.slice.ts  # Authentication state
│   │       └── user.slice.ts   # User profile state
```

- Architectural principles enforced:
- Presentation components do not call APIs directly — they use RTK Query hooks
 - Business logic lives in services, not screens
 - All shared code is in shared/ and never imports from features/
 - Design tokens are imported from design-system/ — no hardcoded values

4. Environment Configuration

File: mobile/src/config/env.ts

```
const API_BASE_URL = process.env.EXPO_PUBLIC_API_BASE_URL ?? 'http://localhost:3002';
const APP_ENV = (process.env.EXPO_PUBLIC_APP_ENV ?? 'development') as 'development' | 'staging' | 'production';

export const config = {
  apiBaseUrl: API_BASE_URL,
  appEnv: APP_ENV,
  isDevelopment: APP_ENV === 'development',
  isProduction: APP_ENV === 'production',
} as const;
```

- How it works:
- All environment variables use the EXPO_PUBLIC_ prefix (required by Expo for client-side access)
 - config is the single source of truth — nothing reads process.env directly
 - Fallback defaults are provided so the app runs without a .env file

Environment file (.env):

```
EXPO_PUBLIC_API_BASE_URL=http://localhost:3002
EXPO_PUBLIC_APP_ENV=development
```

Template (.env.example) is committed to the repository so teammates know what to configure.

5. EAS Build Configuration

File: mobile/eas.json

Three build profiles are configured:

Profile	Distribution	Purpose
development	Internal	Local dev with development client
preview	Internal	Staging builds for QA testing
production	Store	Production release

Each profile injects the correct `EXPO_PUBLIC_APP_ENV` value so the app knows which environment it is running in.

To trigger a build:

```
eas build --profile development --platform ios
eas build --profile development --platform android
```

6. Navigation Foundation

Architecture

Navigation is split into two stacks controlled by authentication state:

```
App.tsx
├── Provider (Redux Store)
│   └── RootNavigator
│       ├── [Loading] → LoadingState ("Restoring session...")
│       ├── [Logged out] → AuthStack
│       │   ├── Login
│       │   ├── ForgotPassword
│       │   └── ResetPassword
│       └── [Logged in] → MainTabs
│           ├── Dashboard
│           ├── Boardrooms
│           ├── Bookings
│           ├── Notifications
│           └── Profile
```

RootNavigator — src/navigation/RootNavigator.tsx

The root navigator is the authentication gate. On mount it:

1. Calls `authService.restoreSession()` to check `AsyncStorage` for a saved token
2. Shows a loading screen while this check is in progress
3. Renders `AuthStack` if not authenticated, `MainTabs` if authenticated

```
export function RootNavigator() {
  const isAuthenticated = useAppSelector(selectIsAuthenticated);
  const isLoading = useAppSelector(selectAuthIsLoading);

  useEffect(() => {
    authService.restoreSession(dispatch).finally(() => {
      dispatch(setAuthLoading(false));
    });
  }, [dispatch]);

  if (isLoading) return <LoadingState message="Restoring session..." />;

  return (
    <NavigationContainer>
      {isAuthenticated ? <MainTabs /> : <AuthStack />}
    </NavigationContainer>
  );
}
```

Protected route handling: Because `MainTabs` is only rendered when `isAuthenticated === true`, unauthenticated users can never reach any main screen. This is enforced at the Redux state level, not at the route level.

AuthStack — src/navigation/AuthStack.tsx

Manages the unauthenticated screens using `createNativeStackNavigator`. Headers are hidden — screens handle their own layout.

MainTabs — src/navigation/MainTabs.tsx

Uses `createBottomTabNavigator`. Tab icons use `Ionicons` with `active/inactive` variants. All colours and font sizes are sourced from the design system.

Type Definitions — src/navigation/types.ts

All navigation is fully typed:

```
export type AuthStackParamList = {
  Login: undefined;
  ForgotPassword: undefined;
  ResetPassword: { token: string }; // token passed as route param
};

export type MainTabParamList = {
  Dashboard: undefined;
  Boardrooms: undefined;
  Bookings: undefined;
  Notifications: undefined;
  Profile: undefined;
};
```

Screen prop types are exported so screens have fully typed navigation and route props.

7. Authentication Foundation

Auth Storage Service — src/features/auth/services/auth-storage.service.ts

Handles all read/write operations to `AsyncStorage`. The token and user profile are stored under namespaced keys.

Key	Value
boardroom:access_token	JWT string
boardroom:user_profile	JSON-serialised user object

Methods:

Method	Description
saveToken(token)	Persists the JWT to AsyncStorage
getToken()	Retrieves the JWT
removeToken()	Removes the JWT
saveUserProfile(profile)	Serialises and saves the user object
getUserProfile<T>()	Deserialises and returns the user object
removeUserProfile()	Removes the user profile
clearAll()	Removes both token and profile (used on logout)

Auth Service — src/features/auth/services/auth.service.ts

Orchestrates authentication state across storage and Redux:

Method	Description
persistLogin(dispatch, token, user)	Saves token + profile to storage, dispatches Redux actions
restoreSession(dispatch)	Reads storage on startup, restores Redux state if token exists
logout(dispatch)	Clears storage, dispatches Redux clear actions

Auth Screens

LoginScreen — src/features/auth/screens/LoginScreen.tsx

- Collects email and password via form
- Validates with loginSchema (Zod)
- Calls useLoginMutation (RTK Query)
- On success: calls authService.persistLogin() → Redux state updates → RootNavigator switches to MainTabs
- On failure: shows native Alert with extracted error message
- Link to ForgotPassword screen

ForgotPasswordScreen — src/features/auth/screens/ForgotPasswordScreen.tsx

- Collects email address
- Validates with forgotPasswordSchema (Zod)
- Calls useForgotPasswordMutation (RTK Query → POST /auth/forgot-password)
- On success: shows SuccessModal then navigates back to Login
- On failure: shows ErrorModal with retry option

ResetPasswordScreen — src/features/auth/screens/ResetPasswordScreen.tsx

- Receives token as a route parameter from the password reset email link
- Collects new password and confirmation
- Validates with resetPasswordSchema including password match check
- Calls useResetPasswordMutation (RTK Query → POST /auth/reset-password)
- On success: shows SuccessModal then navigates to Login
- On failure: shows ErrorModal with retry option

8. API & Data Layer

Base API Slice — src/api/api.slice.ts

Configured using RTK Query's createApi with a fetchBaseQuery base:

```
export const apiSlice = createApi({
  reducerPath: 'api',
  baseQuery: fetchBaseQuery({
    baseUrl: config.apiBaseUrl,
    prepareHeaders(headers, { getState }) {
      const token = (getState() as RootState).auth.accessToken;
      if (token) {
        headers.set('Authorization', `Bearer ${token}`);
      }
      headers.set('Content-Type', 'application/json');
      headers.set('Accept', 'application/json');
      return headers;
    },
  }),
  tagTypes: ['Auth', 'User', 'Boardroom', 'Booking', 'Notification'],
  endpoints: () => ({}),
});
```

Key behaviours:

- Base URL comes from config.apiBaseUrl (environment variable)
- Bearer token is automatically injected on every request from Redux state
- No request requires manual header configuration
- Tag types are pre-registered for cache invalidation in future sprints
- All feature API slices extend this base using injectEndpoints()

Auth API — src/api/auth.api.ts

Extends the base slice with auth endpoints:

Hook	Method	Endpoint	Description
useLoginMutation	POST	/auth/login	Authenticate user
useForgotPasswordMutation	POST	/auth/forgot-password	Request reset email
useResetPasswordMutation	POST	/auth/reset-password	Set new password
useGetProfileQuery	GET	/auth/profile	Fetch current user

Error Handling — src/api/error.utils.ts

extractApiError(error) provides a single standardised function to convert any API error into a human-readable string:

Status	Message Returned
401	"Invalid email or password."
403	"You do not have permission to perform this action."
404	"The requested resource was not found."

409 "A conflict occurred. Please check your input."
5xx "A server error occurred. Please try again."
API error body Uses message field from response body
Unknown "An unexpected error occurred."

All screens use this function — no raw error objects are ever shown to the user.

9. State Management Foundation

Store — src/store/index.ts

```
export const store = configureStore({
  reducer: {
    auth: authReducer,
    user: userReducer,
    [apiSlice.reducerPath]: apiSlice.reducer,
  },
  middleware: (getDefaultMiddleware) =>
    getDefaultMiddleware().concat(apiSlice.middleware),
});
```

Three reducer slices:

Slice	Key	Purpose
authReducer	auth	JWT token, authentication state, loading
userReducer	user	User profile data
apiSlice.reducer	api	RTK Query cache

Typed Hooks — src/store/hooks.ts

```
export const useAppDispatch = () => useDispatch<AppDispatch>();
export const useAppSelector: TypedUseSelectorHook<RootState> = useSelector;
```

All components use useAppDispatch and useAppSelector — never the raw useDispatch/useSelector. This enforces type safety across the entire state layer.

Auth Slice — src/store/slices/auth.slice.ts

State shape:

```
interface AuthState {
  accessToken: string | null;
  isAuthenticated: boolean;
  isLoading: boolean; // true on startup while restoring session
}
```

Actions:

Action	Effect
setCredentials({ accessToken })	Sets token, marks authenticated, clears loading
clearCredentials()	Clears token, marks unauthenticated, clears loading
setAuthLoading(boolean)	Controls loading state during session restore

Selectors:

- selectAccessToken — returns the JWT string
- selectIsAuthenticated — boolean
- selectAuthIsLoading — boolean

User Slice — src/store/slices/user.slice.ts

State shape:

```
interface UserState {
  profile: UserProfile | null;
}
```

Actions:

Action	Effect
setUserProfile(profile)	Stores full user object in state
clearUserProfile()	Clears user profile on logout

Selectors:

- selectUserProfile — full profile object or null
- selectUserFullName — computed "FirstName LastName" string

10. Design System Foundation

All visual properties are defined as typed constants in src/design-system/. No component uses hardcoded values.

Colours — src/design-system/colors.ts

Group	Description
primary.50-900	Brand blue scale (#1E3A5F as primary.500)
neutral.0-900	Grey scale from white to near-black
success	Green — light, main, dark
warning	Amber — light, main, dark
danger	Red — light, main, dark
info	Cyan — light, main, dark
background	App background (#F8F9FA)
surface	Card/panel background (FFFFFF)
border	Default border colour
overlay	Modal backdrop (rgba(0,0,0,0.5))
text.primary/secondary/disabled/inverse/link	Semantic text colours

Typography — src/design-system/typography.ts

Token	Values
fontFamily	System (iOS) / Roboto (Android)
fontSize	xs(10) → 4xl(36)

fontWeight regular(400) → bold(700)
lineHeight tight(1.25) / normal(1.5) / loose(1.75)
letterSpacing tight(-0.5) → wider(1)

Spacing — `src/design-system/spacing.ts`

Scale from 0 to 24 mapping to pixel values (`spacing[4] = 16px`, `spacing[8] = 32px`). All margins, paddings, and gaps use this scale.

Borders — `src/design-system/borders.ts`

Token	Value
<code>borderRadius.sm</code>	4
<code>borderRadius.md</code>	8
<code>borderRadius.lg</code>	12
<code>borderRadius.xl</code>	16
<code>borderRadius.full</code>	9999 (pill shape)
<code>borderWidth.sm/md/lg</code>	1 / 2 / 3

Shadows — `src/design-system/shadows.ts`

Cross-platform shadow factory — uses `shadowColor/Offset/Opacity/Radius` on iOS and `elevation` on Android:

Token	Elevation
<code>shadows.none</code>	None
<code>shadows.sm</code>	2
<code>shadows.md</code>	4
<code>shadows.lg</code>	8
<code>shadows.xl</code>	16

11. Reusable Component Library

All components live in `src/shared/components/` and are exported via barrel files. They are strongly typed, business-logic-free, and exclusively use design system tokens.

Buttons — `src/shared/components/buttons/`

Component	Use Case
<code>PrimaryButton</code>	Main call-to-action. Brand blue background. Supports <code>isLoading</code> spinner.
<code>SecondaryButton</code>	Secondary action. Outlined style.
<code>DangerButton</code>	Destructive action (delete, cancel). Red.
<code>IconButton</code>	Icon-only button for toolbars and compact actions.

All buttons accept `label`, `onPress`, `isLoading`, `disabled`, and `style` props.

Inputs — `src/shared/components/inputs/`

Component	Use Case
<code>TextInput</code>	Standard single-line text field. Supports <code>label</code> , <code>errorMessage</code> , all <code>TextInput</code> props.
<code>PasswordInput</code>	Text input with show/hide password toggle.
<code>SearchInput</code>	Search field with search icon and clear button.
<code>SelectInput</code>	Dropdown/picker input for option selection.

All inputs display validation error messages inline below the field.

Layout — `src/shared/components/layout/`

Component	Use Case
<code>ScreenContainer</code>	Wraps every screen. Provides <code>SafeAreaView</code> , <code>ScrollView</code> , and consistent padding.
<code>PageHeader</code>	Screen title and optional subtitle at the top of a screen.
<code>SectionCard</code>	Elevated card container for grouping related content.

Feedback — `src/shared/components/feedback/`

Component	Use Case
<code>LoadingState</code>	Full-screen spinner with optional message. Used during session restore and data loading.
<code>EmptyState</code>	Illustrated empty view with title, description, and optional action button.
<code>ErrorState</code>	Error view with message and retry button.

Modals — `src/shared/components/modals/`

Component	Props	Use Case
<code>ConfirmationModal</code>	<code>visible</code> , <code>title</code> , <code>message</code> , <code>onConfirm</code> , <code>onCancel</code>	Destructive action confirmation
<code>SuccessModal</code>	<code>visible</code> , <code>title</code> , <code>message</code> , <code>onClose</code>	Success feedback after an action
<code>ErrorModal</code>	<code>visible</code> , <code>message</code> , <code>onClose</code> , <code>onRetry</code>	Error feedback with retry option

All modals use `colors.overlay` for the backdrop — no hardcoded colours.

12. Form Framework

Libraries: `react-hook-form` + `@hookform/resolvers/zod` + `zod`

Validation Schemas — `src/shared/utils/validation.utils.ts`

Reusable primitive schemas:

Schema	Rule
<code>emailSchema</code>	Required, valid email format
<code>passwordSchema</code>	Required, 8–128 characters
<code>requiredStringSchema(fieldName)</code>	Required string, name injected into message
<code>optionalStringSchema</code>	Optional string

Composed form schemas:

Schema	Fields	Used By
<code>loginSchema</code>	<code>email</code> , <code>password</code>	<code>LoginScreen</code>
<code>forgotPasswordSchema</code>	<code>email</code>	<code>ForgotPasswordScreen</code>
<code>resetPasswordSchema</code>	<code>password</code> , <code>confirmPassword</code> (match check)	<code>ResetPasswordScreen</code>

Inferred TypeScript types are exported alongside schemas.

```
export type LoginFormValues = z.infer<typeof loginSchema>;
export type ForgotPasswordFormValues = z.infer<typeof forgotPasswordSchema>;
export type ResetPasswordFormValues = z.infer<typeof resetPasswordSchema>;
```

How Forms Are Wired in Screens

```
// 1. Initialise form with schema resolver
const { control, handleSubmit, formState: { errors } } = useForm<LoginFormValues>({
  resolver: zodResolver(loginSchema),
});

// 2. Connect field to Controller
<Controller
  control={control}
  name="email"
  render={({ field: { onChange, onBlur, value } }) => (
    <TextInput
      value={value}
      onChangeText={onChange}
      onBlur={onBlur}
      errorMessage={errors.email?.message} // inline validation message
    />
  )}
/>

// 3. Submit with validation
<PrimaryButton onPress={handleSubmit(onSubmit)} />
```

Validation runs on submit. Error messages appear inline in the input components. No validation logic exists in screens — it is entirely in `validation.utils.ts`.

13. Notification Framework

File: `src/shared/services/notification.service.ts`

Wraps expo-notifications into a clean service API that screens can consume without importing Expo directly.

Setup: `setNotificationHandler` is configured at module load time to show alerts, play sounds, and update the badge for all incoming notifications.

Methods:

Method	Description
<code>requestPermissions()</code>	Requests notification permission. Returns <code>true</code> if granted.
<code>getPushToken()</code>	Requests permissions, configures Android channel, returns Expo push token.
<code>scheduleLocalNotification(title, body, delaySeconds)</code>	Schedules a local notification with optional delay.
<code>addNotificationReceivedListener(handler)</code>	Subscribes to foreground notification events.
<code>addNotificationResponseListener(handler)</code>	Subscribes to notification tap events.
<code>removeSubscription(subscription)</code>	Cleans up an event subscription.

Permission request is called automatically on app startup in `App.tsx`:

```
useEffect(() => {
  void notificationService.requestPermissions();
}, []);
```

Android channel is configured with maximum importance and the brand vibration pattern when retrieving a push token.

14. Application Entry Point

File: `mobile/App.tsx`

```
export default function App() {
  useEffect(() => {
    void notificationService.requestPermissions();
  }, []);

  return (
    <Provider store={store}>
      <RootNavigator />
    </Provider>
  );
}
```

The entry point does three things only:

1. Requests notification permissions on mount
2. Provides the Redux store to the entire component tree
3. Renders `RootNavigator` which handles all routing

File: `mobile/index.ts`

```
import { registerRootComponent } from 'expo';
import App from './App';
registerRootComponent(App);
```

Expo's `registerRootComponent` ensures the app is correctly registered whether running in Expo Go or as a native build.

15. Authentication Flow — End to End

Login Flow

User enters email + password

- LoginScreen validates with `loginSchema` (Zod)
- `useLoginMutation` sends `POST /auth/login`
- Backend verifies credentials, returns `{ accessToken, user }`
- `authService.persistLogin()`
 - `authStorageService.saveToken(accessToken)` [AsyncStorage]
 - `authStorageService.saveUserProfile(user)` [AsyncStorage]
 - `dispatch(setCredentials({ accessToken }))` [Redux]
 - `dispatch(setUserProfile(user))` [Redux]
- `RootNavigator` detects `isAuthenticated = true`
- Renders `MainTabs` (user is now in the app)

Session Restore Flow (App Reopen)

```
App opens → App.tsx renders → RootNavigator mounts
→ authService.restoreSession(dispatch)
→ authService.getToken() [AsyncStorage read]
→ authService.getUserProfile() [AsyncStorage read]
→ If token + profile found:
    dispatch(setCredentials(...))
    dispatch(setUserProfile(...))
    return true
→ dispatch(setAuthLoading(false))
→ isLoading = false
→ isAuthenticated = true → MainTabs rendered
→ User lands directly in app without logging in again
```

Logout Flow

```
User taps Sign Out (ProfileScreen)
→ authService.logout(dispatch)
→ authService.clearAll() [removes AsyncStorage keys]
→ dispatch(clearCredentials()) [Redux: isAuthenticated = false]
→ dispatch(clearUserProfile()) [Redux: profile = null]
→ RootNavigator detects isAuthenticated = false
→ Renders AuthStack (user sees Login screen)
```

Forgot Password Flow

```
User taps "Forgot your password?" on Login
→ ForgotPasswordScreen
→ Enters email → validated with forgotPasswordSchema
→ useForgotPasswordMutation → POST /auth/forgot-password
→ Backend generates token (32-byte hex), stores with 1-hour expiry
→ Backend sends reset email via MailQueueService (BullMQ)
→ SuccessModal shown: "Check your inbox"
→ User taps Close → navigates to Login
```

Reset Password Flow

```
User opens reset email → clicks link containing token
→ App opens ResetPasswordScreen with token as route param
→ User enters new password + confirmation
→ resetPasswordSchema validates (min 8 chars, passwords match)
→ useResetPasswordMutation → POST /auth/reset-password { token, password }
→ Backend validates token + expiry
→ Backend hashes and saves new password, clears token fields
→ SuccessModal shown: "Password updated"
→ User taps Close → navigates to Login
```

API Request Flow (All Authenticated Requests)

```
Component calls RTK Query hook (e.g. useGetProfileQuery)
→ RTK Query calls fetchBaseQuery
→ prepareHeaders runs:
    reads state.auth.accessToken from Redux
    sets Authorization: Bearer <token>
    sets Content-Type: application/json
→ HTTP request sent to backend
→ Response returned to component via hook
```

16. Acceptance Criteria Checklist

Criteria	Status	Implementation
Application created using React Native Expo	☑	Expo SDK 56, managed workflow
Navigation foundation implemented	☑	RootNavigator, AuthStack, MainTabs
Authentication architecture completed	☑	authService, authService, auth screens
Redux Toolkit configured	☑	store/index.ts with auth + user + api reducers
RTK Query configured	☑	api.slice.ts with fetchBaseQuery
API service layer configured	☑	auth.api.ts extending base slice
Design system established	☑	colors, typography, spacing, borders, shadows
Reusable component library created	☑	16 components across 5 categories
Form infrastructure established	☑	Zod schemas + react-hook-form
Notification framework configured	☑	notification.service.ts
Error handling standardised	☑	extractApiError() used throughout
Application builds successfully	☑	Tested via npx expo start --web
Feature-based folder structure	☑	features/, shared/, navigation/, store/, api/
Environment variable management	☑	.env + config/env.ts
EAS build support	☑	eas.json with dev/preview/production profiles
Protected route handling	☑	Auth state drives navigator via Redux
Persistent session	☑	AsyncStorage restore on startup
No hardcoded colours	☑	All values from design-system/colors.ts
No hardcoded spacing	☑	All values from design-system/spacing.ts
TypeScript strict mode	☑	strict: true in tsconfig.json
No any types	☑	All layers fully typed
Separation of concerns	☑	Screens → Services → API → Store